

Team Assignment 4 Question 2 - Security Architecture & Design Considerations

Admissions Alchemists

Joseph Rissman, Yue Zhang, Nitin Krishna Bojji

Feb 19, 2025

GradAid, as a web application handling user-generated documents and AI-generated content, must address several common security threats at the architecture level. These include injection attacks, such as SQL, NoSQL, and code injection, where malicious code is inserted into the application, potentially compromising data or system integrity. Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF) are also significant risks. XSS involves injecting malicious scripts into web pages viewed by other users, while CSRF tricks users into performing unwanted actions on a website in which they're currently authenticated. Broken authentication and session management can lead to unauthorized access to user accounts and sensitive data. Unauthorized data access, resulting in data breaches, is a major concern, requiring robust access control mechanisms. Finally, Denial-of-Service (DoS) attacks, which flood a system with traffic, disrupting its availability, must be considered.

GradAid's security architecture adopts a layered approach, incorporating best security practices at each level. The frontend, built with Next.js, employs input sanitization to prevent XSS by validating and escaping user input. Content Security Policy (CSP) restricts the sources of executable scripts, further mitigating XSS risks. CSRF protection is implemented using secure tokens, and authentication handling utilizes JSON Web Tokens (JWTs) with HTTP-only cookies for secure session management. The backend, powered by FastAPI, reinforces security with JWT-based authentication and role-based access control (RBAC). Rate limiting is implemented to prevent brute-force attacks on login endpoints, and Pydantic is used for strong API input validation. At the database level, Supabase/PostgreSQL employs encrypted storage for sensitive data, such as passwords, which are hashed using bcrypt. Access control policies, based on RBAC, protect data, and audit logs track database modifications and access patterns. The LangChain LLM engine also has security considerations. Prompt injection protection filters malicious inputs before they reach the LLM, and usage monitoring logs AI interactions to prevent abuse. OpenAI API keys are securely stored using environment variables and server-side requests.

While GradAid faces many security risks as described previously, here are three specific security risks and their corresponding mitigation strategies:

1. **SQL Injection:** Attackers might attempt to inject malicious SQL queries via API inputs, potentially gaining unauthorized access to the database or manipulating data. This risk is mitigated by employing parameterized queries or using an Object-Relational Mapper (ORM) like SQLAlchemy. These techniques prevent direct SQL execution, treating user-provided input as data rather than executable code. By separating the query structure from the user-supplied values, the database can safely interpret and process the request, preventing malicious code injection.
2. **Cross-Site Scripting:** Malicious scripts could be injected into user-generated content, such as within forms or AI prompts, and subsequently executed. This could lead to session

hijacking, data theft, or defacement of the application. Mitigation involves several layers of defense. All user inputs should be thoroughly escaped, converting special characters into their corresponding HTML entities. Enforcing Content Security Policy (CSP) headers restricts the sources from which scripts can be loaded, limiting the impact of any injected scripts. Finally, any HTML generated from user input should be sanitized before rendering, removing potentially harmful tags and attributes.

3. Broken Authentication: Credential leaks, weak passwords, or session hijacking could compromise user accounts, granting attackers access to sensitive information and potentially allowing them to impersonate users. To address this, bcrypt will be used for hashing passwords, ensuring that even if the database is compromised, passwords remain protected. JWT will be employed for secure session management, providing a stateless and verifiable way to authenticate users. Implementing Multi-Factor Authentication (MFA) through OAuth adds an extra layer of security, requiring users to provide multiple forms of verification, making it significantly harder for attackers to gain access even if they have stolen credentials.

Security significantly impacts software maintainability in several key ways. Regular updates for all dependencies, including Next.js, FastAPI, PostgreSQL, and LangChain, are absolutely essential to patch newly discovered vulnerabilities. This necessitates a proactive approach to dependency management, including automated security scans using tools like Snyk or Dependabot. These tools can automatically detect outdated packages with known vulnerabilities, alerting developers to the need for updates and often even suggesting the appropriate update versions. This process reduces the burden of manually tracking security advisories and ensures that the application is running on the most secure versions of its dependencies.

Access control and role management, implemented through RBAC, directly influence maintainability. A well-defined RBAC system simplifies the process of managing user permissions and ensures that users only have access to the resources they absolutely need, adhering to the principle of least privilege. However, maintaining RBAC requires ongoing effort. As the application evolves and new features are added, the access control policies must be reviewed and updated to reflect these changes. This can involve adding new roles, modifying existing permissions, or even restructuring the entire RBAC system. Failing to keep RBAC up-to-date can lead to security gaps or, conversely, overly restrictive permissions that hinder legitimate users.

Handling security incidents presents another significant challenge to maintainability. Robust logging and monitoring systems are essential for detecting suspicious activity, such as unusual login attempts, unexpected data access patterns, or error messages indicative of an attack. These systems provide the information needed to identify and respond to security incidents. However, maintaining these systems requires careful configuration and regular review to ensure they are capturing the necessary data without generating excessive noise.

In conclusion, by implementing a layered security approach, GradAid aims to ensure data integrity, confidentiality, and availability without unduly burdening system maintainability.